

Model theory in LEAN and the Flypitch Project

Based on Han & van Doorn, ITP 2019 & CPP 2020

Section 1

Introduction

The Flypitch Project

- Goal: formalize model theory in Lean 3 and use it to prove the independence of the Continuum Hypothesis (CH)

The Flypitch Project

- Goal: formalize model theory in Lean 3 and use it to prove the independence of the Continuum Hypothesis (CH)
- Two papers:
 - Han & van Doorn, ITP 2019: $\text{ZFC} \not\vdash \text{CH}$
 - Han & van Doorn, CPP 2020: $\text{ZFC} \not\vdash \neg\text{CH}$

Introduction

Goal of this talk

Formalization of model theory and Boolean Valued models in the Flypitch Project VS LEAN and an overview of its application on forcing and the proof of the independence of CH.

Introduction

Goal of this talk

Formalization of model theory and Boolean Valued models in the Flypitch Project VS LEAN and an overview of its application on forcing and the proof of the independence of CH.

The Continuum Hypothesis (CH)

There is no set X with

$$\aleph_0 < |X| < 2^{\aleph_0}.$$

Index

- First order logic and ZFC
- Boolean-valued models
- Mathematical proof of $ZFC \not\vdash CH$ using forcing.

Section 2

First-Order Logic and ZFC

Languages

A *first-order language* $\mathcal{L}$ consists of:

- A set of *function symbols* $\{f_i\}$, each with an arity $n_i \in \mathbb{N}$

Example: $+$ (arity 2), 1 (arity 0)

- A set of *relation symbols* $\{R_j\}$, each with an arity $m_j \in \mathbb{N}$

Example: $<$ (arity 2), $\in X$ (arity 1)

Languages

A *first-order language* $\mathcal{L}$ consists of:

- A set of *function symbols* $\{f_i\}$, each with an arity $n_i \in \mathbb{N}$

Example: $+$ (arity 2), 1 (arity 0)

- A set of *relation symbols* $\{R_j\}$, each with an arity $m_j \in \mathbb{N}$

Example: $<$ (arity 2), $\in X$ (arity 1)

structure Language where

Functions : $\mathbb{N} \rightarrow \text{Type } u$

Relations : $\mathbb{N} \rightarrow \text{Type } v$

Terms

Terms built inductively from variables, constants, and function application. Formally: the smallest set T such that

- every variable x_i is a term
- if f has arity n and $t_1, \dots, t_n \in T$, then $f(t_1, \dots, t_n) \in T$

Terms

Terms built inductively from variables, constants, and function application. Formally: the smallest set T such that

- every variable x_i is a term
- if f has arity n and $t_1, \dots, t_n \in T$, then $f(t_1, \dots, t_n) \in T$

```

inductive preterm : ℕ → Type u
| var {} : ∀ (k : ℕ), preterm 0
| func : ∀ {l : ℕ} (f : L.functions l), preterm l
| app : ∀ {l : ℕ} (t : preterm (l + 1)) (s : preterm 0),
  preterm l
  
```

Flypitch implementation

Terms

Terms built inductively from variables, constants, and function application. Formally: the smallest set T such that

- every variable x_i is a term
- if f has arity n and $t_1, \dots, t_n \in T$, then $f(t_1, \dots, t_n) \in T$

```

inductive preterm : ℕ → Type u
| var {} : ∀ (k : ℕ), preterm 0
| func : ∀ {l : ℕ} (f : L.functions l), preterm l
| app : ∀ {l : ℕ} (t : preterm (l + 1)) (s : preterm 0),
  preterm l

```

Flypitch implementation

```

preterm.app
  (preterm.app
    (preterm.func (L.functions 2).f := f(_, _))
    (preterm.var 1 := x_1))
  (preterm.var 0 := x_0)

```

Terms

```
inductive preterm :  $\mathbb{N} \rightarrow \text{Type } u$   
| var :  $\forall (k : \mathbb{N}), \text{preterm } 0$   
| func :  $\forall \{l : \mathbb{N}\} (f : \text{L.functions } l), \text{preterm } l$   
| app :  $\forall \{l : \mathbb{N}\} (t : \text{preterm } (l + 1)) (s : \text{preterm } 0),$   
  preterm  $l$ 
```

Flypitch implementation

```
inductive Term ( $\alpha : \text{Type } u'$ ) :  $\text{Type } \max u u'$   
| var :  $\alpha \rightarrow \text{Term } \alpha$   
| func :  $\forall \{l : \mathbb{N}\} (_f : \text{L.Functions } l) (_ts : \text{Fin } l \rightarrow$   
  Term  $\alpha), \text{Term } \alpha$ 
```

mathlib implementation

Formulas

Formulas are what we understand as a statement in first-order logic. we built them using terms, relations of the language and the standard logical operations

Formulas

Formulas are what we understand as a statement in first-order logic. we built them using terms, relations of the language and the standard logical operations

```

inductive preformula :  $\mathbb{N} \rightarrow$  Type
| true : preformula 0
| false : preformula 0
| equal : (term L)  $\rightarrow$  (term L)  $\rightarrow$  preformula 0
| rel :  $\forall \{n : \text{nat}\}, \text{L.relations } n \rightarrow$  preformula n
| apprel :  $\forall \{n : \text{nat}\}, \text{preformula } (n + 1) \rightarrow$  (term L)  $\rightarrow$ 
  preformula n
| imp : preformula 0  $\rightarrow$  preformula 0  $\rightarrow$  preformula 0
| all : preformula 0  $\rightarrow$  preformula 0

```


Formulas

Formulas are what we understand as a statement in first-order logic. we built them using terms, relations of the language and the standard logical operations

```

inductive preformula : ℕ → Type
| true : preformula 0
| false : preformula 0
| equal : (term L) → (term L) → preformula 0
| rel : ∀ {n : nat}, L.relations n → preformula n
| apprel : ∀ {n : nat}, preformula (n + 1) → (term L) →
  preformula n
| imp : preformula 0 → preformula 0 → preformula 0
| all : preformula 0 → preformula 0

```

We can write

$$\forall x_0 \forall x_1 (x_0 \in \mathcal{P}(x_1) \iff (\forall x_2 (x_0 \in x_1 \implies x_0 \in x_2)))$$

```

def axiom_of_powerset : sentence L_ZFC :=
  ∀' ∀' (&'0 ∈' P' &'1 <-> (∀' (&'0 ∈' &'1 ⟹ &'0 ∈' &'2)))

```

Formulas

```

inductive BoundedFormula (α : Type u') : ℕ → Type max u
  v u'
| falsum {n} : BoundedFormula α n
| equal {n} (t1 t2 : L.Term (α ⊕ (Fin n))) :
  BoundedFormula α n
| rel {n l : ℕ} (R : L.Relations l)
  (ts : Fin l → L.Term (α ⊕ (Fin n))) : BoundedFormula α
  n
| imp {n} (f1 f2 : BoundedFormula α n) :
  BoundedFormula α n
| all {n} (f : BoundedFormula (n + 1)) :
  BoundedFormula α n

```

In here α indexes the free variables while n indexes the bound variables.

Extra elements

```

notation `⊥` := fol.bounded_preformula.bd_falsum
infix `≈` :88 := fol.bounded_preformula.bd_equal
infixr `⇒` :62 := fol.bounded_preformula.bd_imp
def bd_not {n} (f : bounded_formula L n) :
  bounded_formula L n := f ⇒ ⊥
prefix `~` :max := fol.bd_not
def bd_and {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := ~ (f₁ ⇒ ~ f₂)
infixr `⊓` := fol.bd_and
def bd_or {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := ~ f₁ ⇒ f₂
infixr `⊔` := fol.bd_or
def bd_biimp {n} (f₁ f₂ : bounded_formula L n) :
  bounded_formula L n := (f₁ ⇒ f₂) ⊓ (f₂ ⇒ f₁)
infix `⇔` :61 := fol.bd_biimp
prefix `∀` :110 := fol.bounded_preformula.bd_all
def bd_ex {n} (f : bounded_formula L (n+1)) :
  bounded_formula L n := ~ (∀ ( ~ f))
prefix `∃` :110 := fol.bd_ex

```

Proofs

Proofs are built inductively from axioms and inference rules.

Proofs

Proofs are built inductively from axioms and inference rules.

```

inductive prf : set (formula L) → formula L → Type u
| axm      {Γ A} (h : A ∈ Γ) : prf Γ A
| impI     {Γ} {A B} (h : prf (insert A Γ) B) :
    prf Γ (A ==> B)
| impE     {Γ} (A) {B} (h1 : prf Γ (A ==> B))
    (h2 : prf Γ A) : prf Γ B
| falsumE  {Γ} {A} (h : prf (insert ~A Γ) ⊥) : prf Γ A
| allI     {Γ A} (h : prf ((λ f, f ↑ 1) 00 Γ) A) :
    prf Γ (∀0 A)
| allE2    {Γ} A t (h : prf Γ (∀0 A)) : prf Γ (A[t // 0])
| ref      (Γ t) : prf Γ (t ' t)
| subst2   {Γ} (s t f) (h1 : prf Γ (s ' t))
    (h2 : prf Γ (f[s // 0])) : prf Γ (f[t // 0])
  
```

Proofs

Proofs are built inductively from axioms and inference rules.

```

inductive prf : set (formula L) → formula L → Type u
| axm      {Γ A} (h : A ∈ Γ) : prf Γ A
| impI     {Γ} {A B} (h : prf (insert A Γ) B) :
    prf Γ (A ==> B)
| impE     {Γ} (A) {B} (h1 : prf Γ (A ==> B))
    (h2 : prf Γ A) : prf Γ B
| falsumE  {Γ} {A} (h : prf (insert ~A Γ) ⊥) : prf Γ A
| allI     {Γ A} (h : prf ((λ f, f ↑ 1) 00 Γ) A) :
    prf Γ (∀0 A)
| allE2    {Γ} A t (h : prf Γ (∀0 A)) : prf Γ (A[t // 0])
| ref      (Γ t) : prf Γ (t ' t)
| subst2   {Γ} (s t f) (h1 : prf Γ (s ' t))
    (h2 : prf Γ (f[s // 0])) : prf Γ (f[t // 0])
  
```

Given a set of axioms Γ and a formula A , we define $\Gamma \vdash A$.

```
nonempty (prf Γ A)
```

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

An $\mathcal{L}$ -structure $\mathcal{M}$ gives meaning to the symbols of the language $\mathcal{L}$ by assigning a set M to the domain of discourse and interpreting each function and relation symbol as a function or relation on M .

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

An $\mathcal{L}$ -structure $\mathcal{M}$ gives meaning to the symbols of the language $\mathcal{L}$ by assigning a set M to the domain of discourse and interpreting each function and relation symbol as a function or relation on M .

Example: The structure $\mathcal{M} = (\mathbb{R}, +, 0)$ interprets the symbols of the language of groups $\mathcal{L} = (+, 0)$, $(\mathbb{R}^*, \cdot, 1)$ and $(\mathbb{Z}, +, 0)$ are also $\mathcal{L}$ -structures.

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

An $\mathcal{L}$ -structure $\mathcal{M}$ gives meaning to the symbols of the language $\mathcal{L}$ by assigning a set M to the domain of discourse and interpreting each function and relation symbol as a function or relation on M .

Example: The structure $\mathcal{M} = (\mathbb{R}, +, 0)$ interprets the symbols of the language of groups $\mathcal{L} = (+, 0)$, $(\mathbb{R}^*, \cdot, 1)$ and $(\mathbb{Z}, +, 0)$ are also $\mathcal{L}$ -structures.

```
class Structure (L : Language) (M : Type w) where
  funMap : ∀ {n}, L.Functions n → (Fin n → M) → M := by
    exact fun {n} => isEmptyElim
  RelMap : ∀ {n}, L.Relations n → (Fin n → M) → Prop :=
    by exact fun {n} => isEmptyElim
```

Structures

Formulas are just a collection of symbols. It is require additional information to give them a truth value.

An $\mathcal{L}$ -structure $\mathcal{M}$ gives meaning to the symbols of the language $\mathcal{L}$ by assigning a set M to the domain of discourse and interpreting each function and relation symbol as a function or relation on M .

Example: The structure $\mathcal{M} = (\mathbb{R}, +, 0)$ interprets the symbols of the language of groups $\mathcal{L} = (+, 0)$, $(\mathbb{R}^*, \cdot, 1)$ and $(\mathbb{Z}, +, 0)$ are also $\mathcal{L}$ -structures.

```
class Structure (L : Language) (M : Type w) where
  funMap : ∀ {n}, L.Functions n → (Fin n → M) → M := by
    exact fun {n} => isEmptyElim
  RelMap : ∀ {n}, L.Relations n → (Fin n → M) → Prop :=
    by exact fun {n} => isEmptyElim
```

We write $\mathcal{M} \models A$ if A is true in $\mathcal{M}$.

completeness theorem

Theorem

Let $\mathcal{L}$ be a first-order language, Γ a set of $\mathcal{L}$ -sentences, and A an $\mathcal{L}$ -sentence. Then, $\Gamma \vdash A$ if and only if for any $\mathcal{M}$ such that $\mathcal{M} \models \Gamma$ then $\mathcal{M} \models A$.

Language of ZFC

ZFC is a first-order theory in the language of set theory $\mathcal{L}_\in$ with a single binary relation symbol $\in$.

Language of ZFC

ZFC is a first-order theory in the language of set theory $\mathcal{L}_\in$ with a single binary relation symbol $\in$.

```
inductive ZFC_rel : ℕ → Type
| ε : ZFC_rel 2

def L_ZFC : Language :=
⟨λ_ => Empty, ZFC_rel⟩
```

Axioms of ZFC

ZFC is a first-order theory in the language of set theory $\mathcal{L}_\in$ with a single binary relation symbol $\in$.

```

inductive ZFC_rel : ℕ → Type 1
| ε : ZFC_rel 2

inductive ZFC_func : ℕ → Type 1
| emptyset : ZFC_func 0
| pr : ZFC_func 2
| ω : ZFC_func 0
| P : ZFC_func 1
| Union : ZFC_func 1

def L_ZFC : Language :=
{ functions := ZFC_func,
  relations := ZFC_rel }

```

Language conservatively extended with function symbols $\emptyset, (-, -), \omega, \mathcal{P}(-), \bigcup(-)$ for convenience.

```

axiom_of_emptyset :=  $\forall x, x \notin \emptyset$ 
axiom_of_ordered_pairs :=  $\forall x y z w, (x, y) = (z, w) \leftrightarrow x = z \wedge y = w$ 
axiom_of_extensionality :=  $\forall x y, (\forall z, (z \in x \leftrightarrow z \in y)) \rightarrow x = y$ 
axiom_of_union :=  $\forall u x, x \in \bigcup u \leftrightarrow \exists y \in u, x \in y$ 
axiom_of_powerset :=  $\forall z y, y \in P(z) \leftrightarrow \forall x \in y, x \in z$ 
axiom_of_infinity :=  $\emptyset \in \omega \wedge (\forall x \in \omega, \exists y \in \omega, x \in y) \wedge (\exists \alpha, \text{Ord}(\alpha) \wedge \omega = \alpha) \wedge$ 
     $\forall \alpha, \text{Ord}(\alpha) \rightarrow (\emptyset \in \alpha \wedge \forall x \in \alpha, \exists y \in \alpha, x \in y) \rightarrow \omega \subseteq \alpha$ 
axiom_of_regularity :=  $\forall x, x \neq \emptyset \rightarrow \exists y \in x, \forall z \in x, z \notin y$ 
zorns_lemma :=  $\forall z, z \neq \emptyset \rightarrow (\forall y, (y \subseteq z \wedge \forall x_1 x_2 \in y, x_1 \subseteq x_2 \vee x_2 \subseteq x_1) \rightarrow (\bigcup y) \in z) \rightarrow$ 
     $\exists m \in x, \forall x \in z, m \subseteq x \rightarrow m = x$ 
axiom_of_collection( $\varphi$ ) :=  $\forall p \forall A, (\forall x \in A, \exists y, \varphi(x, y, p)) \rightarrow$ 
     $(\exists B, (\forall x \in A, \exists y \in B, \varphi(x, y, p)) \wedge \forall y \in B, \exists x \in A, \varphi(x, y, p))$ 

epsilon_transitive( $z$ ) :=  $\forall x, x \in z \implies x \subseteq z$ 
epsilon_trichotomy( $z$ ) :=  $\forall x y \in z, x = y \vee x \in y \vee y \in x$ 
epsilon_wellfounded( $z$ ) :=  $\forall x, x \subseteq z \implies x \neq \emptyset \rightarrow \exists y \in x, \forall w \in x, w \notin y$ 
Ord( $z$ ) :=  $\text{epsilon\_trichotomy}(z) \wedge \text{epsilon\_wellfounded}(z) \wedge \text{epsilon\_transitive}(z)$ 

```



```

axiom_of_emptyset :=  $\forall x, x \notin \emptyset$ 
axiom_of_ordered_pairs :=  $\forall x y z w, (x, y) = (z, w) \leftrightarrow x = z \wedge y = w$ 
axiom_of_extensionality :=  $\forall x y, (\forall z, (z \in x \leftrightarrow z \in y)) \rightarrow x = y$ 
axiom_of_union :=  $\forall u x, x \in \bigcup u \leftrightarrow \exists y \in u, x \in y$ 
axiom_of_powerset :=  $\forall z y, y \in P(z) \leftrightarrow \forall x \in y, x \in z$ 
axiom_of_infinity :=  $\emptyset \in \omega \wedge (\forall x \in \omega, \exists y \in \omega, x \in y) \wedge (\exists \alpha, \text{Ord}(\alpha) \wedge \omega = \alpha) \wedge$ 
   $\forall \alpha, \text{Ord}(\alpha) \rightarrow (\emptyset \in \alpha \wedge \forall x \in \alpha, \exists y \in \alpha, x \in y) \rightarrow \omega \subseteq \alpha$ 
axiom_of_regularity :=  $\forall x, x \neq \emptyset \rightarrow \exists y \in x, \forall z \in x, z \notin y$ 
zorns_lemma :=  $\forall z, z \neq \emptyset \rightarrow (\forall y, (y \subseteq z \wedge \forall x_1 x_2 \in y, x_1 \subseteq x_2 \vee x_2 \subseteq x_1) \rightarrow (\bigcup y) \in z)$ 
   $\exists m \in x, \forall x \in z, m \subseteq x \rightarrow m = x$ 
axiom_of_collection( $\varphi$ ) :=  $\forall p \forall A, (\forall x \in A, \exists y, \varphi(x, y, p)) \rightarrow$ 
   $(\exists B, (\forall x \in A, \exists y \in B, \varphi(x, y, p)) \wedge \forall y \in B, \exists x \in A, \varphi(x, y, p))$ 

epsilon_transitive( $z$ ) :=  $\forall x, x \in z \implies x \subseteq z$ 
epsilon_trichotomy( $z$ ) :=  $\forall x y \in z, x = y \vee x \in y \vee y \in x$ 
epsilon_wellfounded( $z$ ) :=  $\forall x, x \subseteq z \implies x \neq \emptyset \rightarrow \exists y \in x, \forall w \in x, w \notin y$ 
Ord( $z$ ) := epsilon_trichotomy( $z$ )  $\wedge$  epsilon_wellfounded( $z$ )  $\wedge$  epsilon_transitive( $z$ )

```

```
def axiom_of_emptyset : Formula L_ZFC :=  $\forall' (\sim(\&0 \in' \emptyset'))$ 
```

pSet

- Lean is based on type theory, not set theory. Ideally we want to encode sets as types.
- Aczel–Werner encoding of sets
- Carneiro added the encoding to mathlib.

pSet

- Lean is based on type theory, not set theory. Ideally we want to encode sets as types.
- Aczel–Werner encoding of sets
- Carneiro added the encoding to mathlib.

```
inductive PSet : Type (u + 1)
| mk (α : Type u) (A : α → PSet) : PSet
```

pSet

- Lean is based on type theory, not set theory. Ideally we want to encode sets as types.
- Aczel–Werner encoding of sets
- Carneiro added the encoding to mathlib.

```
inductive PSet : Type (u + 1)
| mk (α : Type u) (A : α → PSet) : PSet
```

```
def Equiv : PSet → PSet → Prop
| ⟨_, A⟩, ⟨_, B⟩ => (∀ a, ∃ b, Equiv (A a) (B b)) ∧ (∀
  b, ∃ a, Equiv (A a) (B b))
```

Section 3

Boolean-Valued Models

Boolean algebra

Definition

A *Boolean algebra* is a set B with operations $\wedge$ (meet), $\vee$ (join), $\neg$ (complement), and constants $\top, \perp$ satisfying:

- **Commutativity**

$$a \wedge b = b \wedge a$$

- **Associativity**

$$a \wedge (b \wedge c) = (a \wedge b) \wedge c$$

- **Distributivity**

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$$

- **Identity**

$$a \wedge \top = a, \quad a \vee \perp = a$$

- **Complement**

$$a \wedge \neg a = \perp, \quad a \vee \neg a = \top$$

Boolean algebra

Definition

A *Boolean algebra* is a set B with operations $\wedge$ (meet), $\vee$ (join), $\neg$ (complement), and constants $\top, \perp$ satisfying:

- **Commutativity**

$$a \wedge b = b \wedge a$$

- **Associativity**

$$a \wedge (b \wedge c) = (a \wedge b) \wedge c$$

- **Distributivity**

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$$

- **Identity**

$$a \wedge \top = a, \quad a \vee \perp = a$$

- **Complement**

$$a \wedge \neg a = \perp, \quad a \vee \neg a = \top$$

The Partial Order

Define $a \leq b$ iff $a \wedge b = a$. Then $\perp \leq a \leq \top$ for all a .

Complete Boolean Algebras

Definition

A *complete Boolean algebra* $\mathcal{B}$ is a Boolean algebra in which every subset has an infimum $\bigwedge$ and supremum $\bigvee$.

Example

$2 = \{\perp, \top\}$: classical truth values

Example of boolean algebra

Definition

U is *regular open* in space X if $U = (U^\perp)^\perp$, where $U^\perp$ is the interior of the complement of $\overline{U}$. $\text{RO}(X)$ (The regular open sets of X) is a complete Boolean algebra.

Example of boolean algebra

Definition

U is *regular open* in space X if $U = (U^\perp)^\perp$, where $U^\perp$ is the interior of the complement of $\overline{U}$. $\text{RO}(X)$ (The regular open sets of X) is a complete Boolean algebra.

- **Commutativity**

$$a \cup b = b \cup a$$

- **Associativity**

$$a \cup (b \cup c) = (a \cup b) \cup c$$

- **Distributivity**

$$a \cup (b \cap c) = (a \cup b) \cap (a \cup c)$$

- **Identity**

$$a \cap X = a, \quad a \cup \emptyset = a$$

- **Complement**

$$a \cap a^\perp = \emptyset, \quad a \cup a^\perp = X$$

Boolean-valued models

- A *Boolean-valued model* is a structure $\mathcal{M}$ in which formulas are interpreted as elements of a complete Boolean algebra $\mathcal{B}$.

Boolean-valued models

- A *Boolean-valued model* is a structure $\mathcal{M}$ in which formulas are interpreted as elements of a complete Boolean algebra $\mathcal{B}$.
- The truth value of a formula φ in $\mathcal{M}$ is denoted by $\llbracket \varphi \rrbracket_{\mathcal{M}} \in \mathcal{B}$.
- The interpretation of logical connectives and quantifiers is given by the operations of the Boolean algebra:

$$\llbracket \varphi \wedge \psi \rrbracket_{\mathcal{M}} = \llbracket \varphi \rrbracket_{\mathcal{M}} \sqcap \llbracket \psi \rrbracket_{\mathcal{M}}$$

$$\llbracket \varphi \vee \psi \rrbracket_{\mathcal{M}} = \llbracket \varphi \rrbracket_{\mathcal{M}} \sqcup \llbracket \psi \rrbracket_{\mathcal{M}}$$

$$\llbracket \neg \varphi \rrbracket_{\mathcal{M}} = \neg \llbracket \varphi \rrbracket_{\mathcal{M}}$$

$$\llbracket \forall x \varphi(x) \rrbracket_{\mathcal{M}} = \bigcap_m \llbracket \varphi(m) \rrbracket_{\mathcal{M}}$$

$$\llbracket \exists x \varphi(x) \rrbracket_{\mathcal{M}} = \bigcup_m \llbracket \varphi(m) \rrbracket_{\mathcal{M}}$$

Boolean-valued models

```
structure bStructure :=
  (carrier : Type u)
  (fun_map :  $\forall\{n\}$ , L.functions  $n \rightarrow$  dvector carrier  $n \rightarrow$ 
    carrier)
  (rel_map :  $\forall\{n\}$ , L.relations  $n \rightarrow$  dvector carrier  $n \rightarrow \beta$ )
  (eq : carrier  $\rightarrow$  carrier  $\rightarrow \beta$ )
  (eq_refl :  $\forall x$ , eq  $x x = \top$ )
  (eq_symm :  $\forall x y$ , eq  $x y = eq y x$ )
  (eq_trans :  $\forall\{x\} y \{z\}$ , eq  $x y \sqcap eq y z \leq eq x z$ )
  (fun_congr :  $\forall\{n\}$  (f : L.functions  $n$ ) (x y : dvector
    carrier  $n$ ),
    (x.map2 eq y).fInf  $\leq eq$  (fun_map f x) (fun_map f y))
  (rel_congr :  $\forall\{n\}$  (R : L.relations  $n$ ) (x y : dvector
    carrier  $n$ ),
    (x.map2 eq y).fInf  $\sqcap rel\_map R x \leq rel\_map R y$ )
```

Boolean-Valued Soundness Theorem

Theorem

If $\Gamma \vdash \varphi$, then for every $\mathcal{B}$ and every $\mathcal{B}$ -valued structure $\mathcal{M}$:

$$\bigwedge_{\gamma \in \Gamma} \llbracket \gamma \rrbracket_{\mathcal{M}} \leq \llbracket \varphi \rrbracket_{\mathcal{M}}.$$

Why is this useful?

- $\mathcal{M} \models_{\mathcal{B}} \gamma$ for $\gamma \in \text{ZFC}$, $\llbracket \text{CH} \rrbracket_{\mathcal{M}} < \top \implies \text{ZFC} \not\vdash \text{CH}$
- $\mathcal{M} \models_{\mathcal{B}} \gamma$ for $\gamma \in \text{ZFC}$, $\llbracket \neg \text{CH} \rrbracket_{\mathcal{M}} < \top \implies \text{ZFC} \not\vdash \neg \text{CH}$

Proved by structural induction on proof trees.

From pSet to bSet

Aczel–Werner encoding of V :

```
inductive pSet : Type (u+1)
| mk (a : Type u) (A : a -> pSet) : pSet
```

From pSet to bSet

Aczel–Werner encoding of V :

```
inductive pSet : Type (u+1)
| mk (a : Type u) (A : a -> pSet) : pSet
```

Generalize with a third field assigning Boolean truth-values:

```
inductive bSet (B : Type u) [cBA B] : Type (u+1)
| mk (a : Type u) (A : a -> bSet) (b : a -> B) : bSet
```


Boolean-Valued Equality and Membership

Defined with implementation:

```
def bv_eq'' : ∀ (x y : bSet B), B
| (α, A, B) (α', A', B') :=
| | | | | (⊓a : α, B a ⇒ ⊔a', B' a' ∩ bv_eq (A a) (A' a')) ∩
| | | | | (⊓a' : α', B' a' ⇒ ⊔a, B a ∩ bv_eq (A a) (A' a'))

def mem : bSet B → bSet B → B
| a (mk α' A' B') := ⊔a', B' a' ∩ a =B A' a'
```

Boolean-Valued Equality and Membership

Defined with implementation:

```
def bv_eq' : ∀ (x y : bSet B), B
| (α, A, B) (α', A', B') :=
| | | | | (⊓a : α, B a ⇒ ⊔a', B' a' ⊓ bv_eq (A a) (A' a')) ⊓
| | | | | (⊓a' : α', B' a' ⇒ ⊔a, B a ⊓ bv_eq (A a) (A' a'))

def mem : bSet B → bSet B → B
| a (mk α' A' B') := ⊔a', B' a' ⊓ a =B A' a'
```

We denote this structure by $\mathbf{V}^{\mathcal{B}}$.

Theorem (Fundamental Theorem of Forcing)

For every complete Boolean algebra $\mathcal{B}$, $\mathbf{V}^{\mathcal{B}} \models_{\mathcal{B}} \text{ZFC}$: every axiom σ of ZFC has $\llbracket \sigma \rrbracket_{\mathbf{V}}^{\mathcal{B}} = \top$.

Section 4

The proof

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathcal{B}}$)

Let $\mathcal{B} = \text{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathcal{B}}$ be the corresponding Boolean-valued universe. Then

$$\llbracket \neg \text{CH} \rrbracket_{\mathcal{B}} = \top.$$

That is, $\mathbf{V}^{\mathcal{B}}$ satisfies $\neg \text{CH}$.

Solution.

$\neg \text{CH}$ asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathcal{B}}$ witnesses this with $\aleph_1$.

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathcal{B}}$)

Let $\mathcal{B} = \text{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathcal{B}}$ be the corresponding Boolean-valued universe. Then

$$\llbracket \neg \text{CH} \rrbracket_{\mathcal{B}} = \top.$$

That is, $\mathbf{V}^{\mathcal{B}}$ satisfies $\neg \text{CH}$.

Solution.

$\neg \text{CH}$ asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathcal{B}}$ witnesses this with $\check{\aleph}_1$.

Claim 1: $\llbracket \check{\omega} \text{ is strictly smaller than } \check{\aleph}_1 \rrbracket = \top$.

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathcal{B}}$)

Let $\mathcal{B} = \text{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathcal{B}}$ be the corresponding Boolean-valued universe. Then

$$\llbracket \neg \text{CH} \rrbracket_{\mathcal{B}} = \top.$$

That is, $\mathbf{V}^{\mathcal{B}}$ satisfies $\neg \text{CH}$.

Solution.

$\neg \text{CH}$ asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathcal{B}}$ witnesses this with $\check{\aleph}_1$.

Claim 1: $\llbracket \check{\omega} \text{ is strictly smaller than } \check{\aleph}_1 \rrbracket = \top$.

Claim 2: $\llbracket \check{\aleph}_1 \text{ is strictly smaller than } \check{\aleph}_2 \rrbracket = \top$.

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathcal{B}}$)

Let $\mathcal{B} = \text{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathcal{B}}$ be the corresponding Boolean-valued universe. Then

$$\llbracket \neg \text{CH} \rrbracket_{\mathcal{B}} = \top.$$

That is, $\mathbf{V}^{\mathcal{B}}$ satisfies $\neg \text{CH}$.

Solution.

$\neg \text{CH}$ asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathcal{B}}$ witnesses this with $\check{\aleph}_1$.

Claim 1: $\llbracket \check{\omega} \text{ is strictly smaller than } \check{\aleph}_1 \rrbracket = \top.$

Claim 2: $\llbracket \check{\aleph}_1 \text{ is strictly smaller than } \check{\aleph}_2 \rrbracket = \top.$

Claim 3: $\llbracket \check{\aleph}_2 \hookrightarrow \mathcal{P}(\check{\omega}) \rrbracket = \top.$

Sketch of the proof

Theorem (Failure of CH in $\mathbf{V}^{\mathcal{B}}$)

Let $\mathcal{B} = \text{RO}(2^{\aleph_2 \times \omega})$ and let $\mathbf{V}^{\mathcal{B}}$ be the corresponding Boolean-valued universe. Then

$$\llbracket \neg \text{CH} \rrbracket_{\mathcal{B}} = \top.$$

That is, $\mathbf{V}^{\mathcal{B}}$ satisfies $\neg \text{CH}$.

Solution.

$\neg \text{CH}$ asserts the existence of a set strictly between ω and $\mathcal{P}(\omega)$ in cardinality. We show $\mathbf{V}^{\mathcal{B}}$ witnesses this with $\check{\aleph}_1$.

Claim 1: $\llbracket \check{\omega} \text{ is strictly smaller than } \check{\aleph}_1 \rrbracket = \top.$

Claim 2: $\llbracket \check{\aleph}_1 \text{ is strictly smaller than } \check{\aleph}_2 \rrbracket = \top.$

Claim 3: $\llbracket \check{\aleph}_2 \hookrightarrow \mathcal{P}(\check{\omega}) \rrbracket = \top.$

$$\llbracket \check{\omega} \prec \check{\aleph}_1 \prec \check{\aleph}_2 \leq \mathcal{P}(\check{\omega}) \rrbracket_{\mathcal{B}} = \top,$$

Main theorem

Theorem (Unprovability of CH)

If ZFC is consistent, then $\text{ZFC} \not\vdash \text{CH}$. Equivalently, $\text{ZFC} + \neg\text{CH}$ is consistent relative to ZFC.

Solution.

Assume ZFC is consistent. We have established:

- ① We know $\mathbf{V}^{\mathcal{B}}$ is a $\mathcal{B}$ -valued model of ZFC: $\llbracket \sigma \rrbracket_{\mathcal{B}} = \top$ for every axiom σ of ZFC.
- ② and by the previous theorem, $\llbracket \neg\text{CH} \rrbracket_{\mathcal{B}} = \top$.

By the Boolean-valued soundness theorem, if $\text{ZFC} \vdash \text{CH}$, then

$$\top = \bigcap_{\sigma \in \text{ZFC}} \llbracket \sigma \rrbracket_{\mathcal{B}} \leq \llbracket \text{CH} \rrbracket_{\mathcal{B}}.$$

But $\llbracket \text{CH} \rrbracket_{\mathcal{B}} \leq \neg \llbracket \neg\text{CH} \rrbracket_{\mathcal{B}} = \neg \top = \perp$, a contradiction. Hence $\text{ZFC} \not\vdash \text{CH}$.

Section 5

Ending

Why Boolean-Valued Models

- Avoid reflection theorem, Löwenheim–Skolem, Mostowski collapse, genericity, countable transitive models, the forcing relation and its truth/definability lemmas.
- Both arguments can be built with the same framework: $\mathcal{B}$ -valued logic, $\mathcal{B}$ -valued set theory, and the fundamental theorem of forcing.
- `bSet` is an inductive type (in a very Lean way) and avoids transfinite recursion and the quotient construction of `pSet`.

Thank you

Any questions?

Bibliography

- [1] Peter Aczel et al. “The type theoretic interpretation of constructive set theory”. In: *Journal of Symbolic Logic* 49.1 (1984).
- [2] Jesse Michael Han and Floris van Doorn. “A formal proof of the independence of the continuum hypothesis”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. 2020, pp. 353–366.
- [3] Jesse Michael Han and Floris van Doorn. “A formalization of forcing and the unprovability of the continuum hypothesis”. In: *arXiv preprint arXiv:1904.10570* (2019).
- [4] Richard Mansfield and John Dawson. “Boolean-valued set theory and forcing”. In: *Synthese* 33.1 (1976), pp. 223–252.
- [5] *mathlib4 documentation*. Lean community mathlib4 reference documentation. URL: https://leanprover-community.github.io/mathlib4_docs (visited on 06/07/2026).